

Distributed-systems patterns (the recurring toolkit)

How to read this file: PART 1 = general patterns that recur across every backend system. PART 2 = where each shows up in WCX. PART 3 = takeaways. This file is the "why is the same idea everywhere" consolidation. **Why this file exists:** the same handful of principles — statelessness, idempotency, verify-don't-blind-retry, policy-vs-mechanism, "this repo consumes a managed-service artifact" — recurred across every other note. Naming them once, generally, is more valuable than re-deriving them per topic. **Related:** [\[\[kb-storage-model\]\]](#) (statelessness/where state lives), [\[\[kb-streaming-pipeline\]\]](#) (idempotent send, snapshot polling), [\[\[kb-agent-loop\]\]](#), [\[\[kb-concurrency-async\]\]](#), [\[\[kb-auth-infra\]\]](#), [\[\[kb-rag-retrieval\]\]](#).

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

Deep principle in one line: once a system spans more than one machine (or one process and one network hop), *nothing is reliable by default* — messages get lost, duplicated, reordered, and delayed — so correctness comes from patterns that assume failure, not from hoping it won't happen.

Why these patterns exist / what they solve. A single-process program can trust that a function call completes exactly once and its result is intact. The moment a network sits between two components, every one of those guarantees evaporates: a request may arrive twice, its response may vanish, a dependency may be down, two callers may race. The recurring patterns in this file — statelessness, idempotency, retries-with-backoff, caching, policy/mechanism separation, the managed-service boundary — are the accumulated industry answers to "how do I stay correct and fast when the pieces are unreliable and distributed?" They aren't six random tricks; they're facets of one worldview: **design for failure and scale as first-class concerns, not afterthoughts.**

Real-world connection beyond this repo. This is backend engineering at scale. Every microservice architecture, every cloud app, every queue/event system embodies these: statelessness is why Kubernetes can kill and reschedule your pod freely; idempotency keys are why Stripe won't double-charge you on a retry; at-least-once + idempotent consumers is the default contract of Kafka/SQS/PubSub; retries-with-jitter and circuit breakers are in every resilient client library (and are what a "chaos engineering" team stress-tests); policy-vs-mechanism is why feature flags and config-driven behavior exist. These are also the *most common senior/staff system-design interview themes* — "how do you make this idempotent?", "what happens if this call is retried?", "where does the state live?".

What breaks without these patterns:

- **Non-idempotent writes + retries** → duplicate orders/charges/messages (the canonical distributed bug).
- **Stateful servers** → you can't scale horizontally, can't load-balance freely, and a crash loses in-flight work.
- **No timeout/backoff** → one slow dependency cascades into a full outage (thread pools exhaust; retry storms hammer a recovering service — the "thundering herd").
- **Cache keyed by user-supplied input** → cross-user data leaks (a real security hole, see the OBO gem in [\[\[kb-auth-infra\]\]](#)).
- **Policy hardcoded in code** → every rule change needs a deploy + review, so behavior ossifies.
- **Not knowing the managed-service boundary** → you waste hours trying to debug (or "fix") something that lives entirely on the vendor's side of a REST call.

The transferable mental model. For any component ask: *what happens if this call is retried? if it's lost? if this instance dies right now? if two of these run at once?* If you have a crisp answer to each, you've applied these patterns. The unifying instinct: **push state to one durable place, make operations safe to repeat, assume every remote call can fail, and separate the rules that change (config) from the engine that runs them (code).**

PART 1 — GENERAL KNOWLEDGE

1.1 Statelessness (push state to one place)

Keep servers stateless; externalize state to ONE durable store. Buys horizontal scaling, load-balancer freedom, crash-resilience (see [\[\[kb-storage-model\]\]](#) §1.1). The recurring move: whenever a layer *could* hold state, ask "can I push this to the single source of truth and keep this layer stateless?" LLMs, HTTP, and MCP servers are all deliberately stateless for this reason — state lives in the conversation store, not the compute.

1.2 Idempotency (the same operation twice = same effect as once)

An operation is **idempotent** if doing it multiple times has the same effect as doing it once. Critical because networks are unreliable: a request can succeed on the server but its **response gets lost**, leaving the client unsure whether to retry. Blind retry of a non-idempotent operation (e.g. "post message", "charge card") → duplicates.

- **HTTP baseline:** GET/PUT/DELETE are meant to be idempotent; POST is not (each POST creates a new thing).
- **Idempotency keys:** attach a unique client-generated key to a mutating request; the server dedupes by that key so a retry with the same key is a no-op. (Stripe's classic pattern.)
- **Verify-then-retry** (when you don't control the server's dedupe): before retrying a possibly-succeeded write, *check whether it landed* (query for its effect); only retry if it genuinely didn't. Never blind-retry a create.

1.3 Delivery guarantees

- **At-most-once:** never duplicated, may be lost (fire-and-forget).
- **At-least-once:** never lost, may be duplicated → **consumers must be idempotent** to tolerate the dupes.
- **Exactly-once:** the holy grail; truly achieved only via idempotency + dedup on top of at-least-once (there's no magic — "exactly-once delivery" is mostly "at-least-once delivery + idempotent processing"). Most real systems choose at-least-once + idempotent handlers.

1.4 Retries, timeouts, backoff, circuit breakers

- **Timeouts:** every remote call needs one; a hung dependency must not hang you forever.
- **Retries with exponential backoff + jitter:** retry transient failures, but increase the delay each time (and randomize it) so a recovering service isn't hammered by a synchronized retry storm ("thundering herd").
- **Circuit breaker:** after N consecutive failures, stop calling a dead dependency for a cooldown (fail fast) rather than piling on.
- **Only retry idempotent operations** (§1.2) — or you duplicate.

1.5 Caching (and its two hard problems)

Cache = keep a fast local copy of expensive-to-fetch data. Hard problems: **invalidation** (when is the copy stale?) and **cache key design** (what identifies an entry?).

- **Lazy singleton / read-through cache:** "in cache? return it : build once, store, return." Used for expensive-to-construct clients (auth handshakes) — see [[kb-rag-retrieval]].
- **TTL:** entries expire after a time window (handles freshness).
- **Cache-key SECURITY:** the key must be a **server-verified** identity, never a user-supplied string — else a crafted/colliding key could fetch another user's cached data (cross-user leak). See the OBO cache-key lesson in [[kb-auth-infra]]. Freshness (TTL) and isolation (key choice) are *separate* concerns.
- **Double-checked locking:** build-once-under-concurrency for a shared cache entry (see [[kb-concurrency-async]] §1.7).

1.6 Separation of policy vs mechanism

Policy = the rules that change often (which incident→which flow, which product→which index). **Mechanism** = the engine that evaluates them, rarely changes. Keep policy in **data/config** (YAML, a table, a feature flag), mechanism in **code**. Payoff: change behavior *without a code change* → *without redeploy* → *without eng review*; non-engineers can safely edit the policy; the stable engine stays locked. "Easier to read" is a symptom, not the reason. (Feature flags are this pattern for on/off ops control without redeploy.)

1.7 Service boundaries & the "managed-service consumer" pattern

Systems are increasingly assembled from **managed services** (search, auth, LLM agents, queues) you *consume* rather than *build*. A recurring realization: **your repo is the CONSUMER of a pre-built artifact; the CONSTRUCTION lives across the service boundary.**

- You configure a **pointer** (which index / which agent name / which thread id); the service owns the **substance** (the index contents / the agent's instructions / the transcript).
- **How to detect the boundary:** grep for the construction code (indexer/ingest/training). Absence is evidence it's managed elsewhere. What you *can* see is usually the query/read side + a reference/name/id.
- Implication for debugging: when something's "wrong," first decide which side of the boundary owns it. You can't fix (or even see) the managed side from your repo — you inspect it via its portal/API.

1.8 Adapter / wrapper pattern at boundaries

Wrap an external SDK/service in your own thin class to: simplify its interface, centralize logging/timing/error-handling, and make it swappable (change providers → edit one file). The wrapper is the **seam** between your code and the vendor's. Related: translating

between two systems' vocabularies (an event stream → SSE frames) is an adapter too. (See the `SearchClient` wrapper and `NonClosingCredential` in [\[\[kb-rag-retrieval\]\]](#)/[\[\[kb-auth-infra\]\]](#).)

1.9 "Extract loosely, validate strictly" / trust boundaries

Validate at every boundary you don't control. The client UI blocking bad input is cosmetic — anyone can hit the API directly (curl), so the server MUST re-validate. Corollary for streaming: validate *before* you open the stream, because once headers + first frame are on the wire you can't cleanly return an error (a one-way door).

PART 2 — WORKED EXAMPLE: these patterns in WCX

- **Statelessness (§1.1):** LLM, MCP server (`stateless_http=True`), and HTTP layer are all stateless; conversation state lives only in the SRE **thread**. The portal host "stays stateless about content" — it holds just the `thread_id` . See [\[\[kb-storage-model\]\]](#).
 - **Idempotency via verify-then-retry (§1.2):** `_send_follow_up` — on a lost `post_message` response it VERIFIES whether a new-since-baseline user message matching the text landed, instead of blind-retrying → never duplicates a turn. `baseline_ids` is the "before" reference. See [\[\[kb-streaming-pipeline\]\]](#).
 - **Snapshot polling + at-least-once feel (§1.3):** the diff engine tolerates re-seeing the same message across polls (the `_emitted_text_len` bookmark makes reprocessing idempotent — re-emitting is suppressed).
 - **Timeouts & liveness (§1.4):** `_MAX_STREAM_SECONDS` , `_FIRST_RESPONSE_TIMEOUT` , heartbeats, 90s client silence-abort.
 - **Caching (§1.5):** `ResourceManager` lazy-singleton client cache keyed by (`service_name` , `index_name`) with double-checked locking; **OBO credential cache keyed by the verified `oid` , never the raw user token** (cross-user isolation) — the security gem. See [\[\[kb-auth-infra\]\]](#).
 - **Policy vs mechanism (§1.6):** `routing.yaml` (incident→flow) and prompt-fragment flows are DATA; `_matches` / `compose` are the stable engine. Non-engineers can edit routing without a redeploy. The `get_hub_text_streaming()` feature flag flips behavior without redeploy.
 - **Managed-service consumer boundary (§1.7):** recurs everywhere — the SRE **thread** (Azure owns the transcript; repo holds `thread_id`), the **agent instructions** (uploaded once, the service injects them; repo sends only the agent NAME), the **search index** (repo holds the index NAME; Azure's data source/indexer/skillset build it — proven by grep-absence). See [\[\[kb-rag-retrieval\]\]](#), [\[\[kb-agent-loop\]\]](#).
 - **Adapter/wrapper (§1.8):** the repo's `SearchClient` wraps Azure's; `NonClosingCredential` wraps a shared credential; the old Foundry streamer adapted events→SSE.
 - **Trust boundary (§1.9):** fail-fast message validation happens *before* `stream()` opens the SSE response, because streaming is a one-way door.
-

PART 3 — transferable takeaways

1. Keep compute stateless; push state to one durable store — scale + resilience follow.
2. Networks lose responses → make mutations idempotent (keys) or verify-then-retry; never blind-retry a create.
3. Prefer at-least-once + idempotent consumers over chasing "exactly-once."
4. Every remote call: timeout + backoff-with-jitter + (optionally) circuit breaker; only retry idempotent ops.
5. Cache keys must be server-verified identities (isolation) and TTL'd (freshness) — separate concerns.
6. Put fast-changing rules in config (policy), the engine in code (mechanism) → change behavior without redeploy.
7. Know which side of a managed-service boundary owns a thing; you consume a pointer, the service owns the substance.
8. Validate at every boundary you don't control; for streaming, validate before the one-way door opens.